

Galaxy

W1: Which coding exon has the highest number of single nucleotide polymorphisms (SNPs) on human chromosome 22?

Instruction Only Prompt: Create a Galaxy workflow that identifies the coding exon with the highest number of single-nucleotide polymorphisms (SNPs) on human chromosome 22. Include all necessary tools, input data types, and expected outputs.

Response from gpt-4o:

Here is a step-by-step Galaxy workflow that fulfills the task: **Identify the coding exon with the highest number of SNPs on human chromosome 22**. This response follows your prompt and includes the **necessary tools**, **input formats**, and **expected outputs**.

Galaxy Workflow: SNP Count per Coding Exon on Chromosome 22

♦ Objective

Identify which coding exon on human chromosome 22 contains the highest number of single-nucleotide polymorphisms (SNPs).

Step-by-Step Workflow

Step	Tool	Purpose	Input Format	Output Format
1	Upload Data	Import exon annotations (BED) and SNP positions (BED) for chromosome 22	.bed files	.bed
2	Filter exon BED (if full genome provided)	Extract only chromosome 22 exons	Text Filter or Select lines	.bed
3	Filter SNP BED (if full genome provided)	Extract SNPs on chromosome 22	Text Filter or Select lines	.bed

4	Join (Operate on Genomic Intervals → Join)	Find overlapping regions between exons and SNPs	Exons BED + SNPs BED	Table with exon-SNP overlaps
5	Group (Group on exon ID)	Count number of SNPs per exon	Joined table	Table: exon ID + SNP count
6	Sort (Sort by SNP count)	Rank exons by SNP count in descending order	Grouped table	Sorted table
7	Select First	Extract the exon with the highest SNP count	Sorted table	Single exon ID
8	Join (again with original exon BED)	Retrieve full coordinates of the top exon	Top exon ID + Exon BED	BED format with full annotation
9	Cut Columns / Convert to BED6 (if needed)	Format for visualization or export	Joined table	.bed or .tsv
10	Visualize or Export	Optional: View in UCSC Genome Browser or download	.bed or .csv	UCSC link / Local file

Input Requirements

- **Exon annotations:** BED file of coding exons (from UCSC Table Browser or Gencode).
- **SNP dataset:** BED file of SNP coordinates (from dbSNP or UCSC).

Expected Output

- A single row representing the **exon with the highest number of SNPs**, including:
 - Chromosome
 - Start/End positions
 - Exon ID
 - SNP count

Workflow Export

Once validated, this history can be extracted as a Galaxy workflow using “**Extract Workflow**”, allowing reuse across datasets and organisms.

Response from Gemini 2.5 Flash:

Galaxy Workflow: Identifying the Coding Exon with the Highest Number of SNPs on Human Chromosome 22

This workflow outlines the steps and tools required within the Galaxy platform to answer the question: "Which coding exon has the highest number of single nucleotide polymorphisms (SNPs) on human chromosome 22?"

Workflow Objective

To systematically process genomic data (coding exons and SNPs) for human chromosome 22, identify SNPs within coding exons, count them, and ultimately determine the coding exon with the maximum number of associated SNPs.

Step-by-Step Workflow

Step 1: Obtain Coding Exons for Human Chromosome 22

- **Tool:** UCSC Table Browser (This is usually an external data source integrated into Galaxy under "Get Data" or a tool that can query it)
- **Input Data:** None (initial data retrieval)
 - **Settings:**
 - **Genome:** Human
 - **Assembly:** hg19 or hg38 (specify based on availability and preference, hg38 is more current)
 - **Group:** Genes and Gene Predictions
 - **Track:** RefSeq Genes (or similar, providing exon annotations)
 - **Region:** Position: chr22
 - **Output format:** BED (Browser Extensible Data)
 - **Table:** refGene
- **Action:** Downloads the genomic coordinates and annotations for all RefSeq gene exons on human chromosome 22.
- **Output Data:**
 - **Type:** BED file
 - **Content:** List of all exons on chr22, including their start/end positions, strand,

gene name, and exon rank.

Step 2: Obtain Single Nucleotide Polymorphisms (SNPs) for Human Chromosome 22

- **Tool:** UCSC Table Browser (or a similar data source like Ensembl or dbSNP data if directly available)
- **Input Data:** None (initial data retrieval)
 - **Settings:**
 - **Genome:** Human
 - **Assembly:** hg19 or hg38 (must match Step 1)
 - **Group:** Variation and Repeats
 - **Track:** dbSNP (or Common SNPs)
 - **Region:** Position: chr22
 - **Output format:** BED (or similar tabular format that can be converted)
 - **Table:** snp150Common (or the most recent common SNP table)
- **Action:** Downloads the genomic coordinates and IDs for common SNPs on human chromosome 22.
- **Output Data:**
 - **Type:** BED file (or tabular data convertible to BED)
 - **Content:** List of SNPs on chr22, including their start/end positions (usually a 1bp interval), and dbSNP ID.

Step 3: Extract Coding Exons

- **Tool:** Filter Data on values of Datasets (or a specific "Filter" tool if available) or Text manipulation tools if BED format requires parsing.
- **Input Data:** Output from Step 1 (Exons on chr22)
 - **Settings (Conceptual):**
 - If the BED file from UCSC directly indicates coding status, filter based on a specific column.
 - Alternatively, if the initial exon data includes gene models, you might need a tool that can identify coding sequences (CDS) within those gene models. For RefSeq, coding exons are usually within the 'CDS' portion of the transcript. You might need a tool like Intersect with a separate CDS track or a more advanced gene annotation parser.
 - *Simplified assumption for this tutorial:* The initial exon list implicitly contains coding exons, or we'd refine it further with another annotation source. For true coding exons, you might need to fetch coding regions instead of just exons from UCSC or use a tool like BEDTools Intersect with

CDS regions. For simplicity, we'll assume the RefSeq Exons list contains the necessary information to distinguish coding regions or we focus on a subset.

- **Action:** Refines the list of exons to include only those that are coding. This often involves parsing the 6th or 9th column of a BED file or linking to a separate CDS track.
- **Output Data:**
 - **Type:** BED file
 - **Content:** A filtered list of only coding exons on chr22.

Step 4: Intersect SNPs with Coding Exons

- **Tool:** BEDTools intersect (or "Operate on Genomic Intervals > Intersect")
- **Input Data:**
 - Dataset A: Output from Step 3 (Coding Exons)
 - Dataset B: Output from Step 2 (SNPs on chr22)
- **Action:** Finds all SNPs that physically overlap with any of the coding exons.
- **Output Data:**
 - **Type:** BED file
 - **Content:** A new BED file where each line represents an instance of a SNP falling within a coding exon. The output usually includes the original fields of both the exon and the SNP that overlapped.

Step 5: Count SNPs per Coding Exon

- **Tool:** Group (or "Text manipulation > Count occurrences")
- **Input Data:** Output from Step 4 (Intersected SNPs and Coding Exons)
- **Settings:**
 - **Group by:** The unique identifier for each coding exon (e.g., a combination of chromosome, start, end, and strand from the exon fields).
 - **Operation:** Count the number of rows for each group.
- **Action:** Aggregates the intersected data, counting how many SNPs are associated with each unique coding exon.
- **Output Data:**
 - **Type:** Tabular file
 - **Content:** Two columns: 1) Unique Exon Identifier, 2) Number of SNPs found in that exon.

Step 6: Find the Exon with the Maximum SNP Count

- **Tool:** Sort data in ascending or descending order (or "Text manipulation > Sort")

- **Input Data:** Output from Step 5 (Exon SNP Counts)
- **Settings:**
 - **Sort by:** The column containing the SNP count.
 - **Order:** Descending.
- **Action:** Sorts the list of coding exons based on their SNP count from highest to lowest.
- **Output Data:**
 - **Type:** Tabular file
 - **Content:** The same two-column data as Step 5, but sorted. The first row will represent the coding exon with the highest number of SNPs.

Expected Final Output

The final output will be a sorted tabular file. The very first row of this file will contain the details of the coding exon on human chromosome 22 that has the highest number of SNPs, along with its corresponding SNP count. This row directly answers the posed question.

This workflow structure can be built and executed within the Galaxy environment, allowing for reproducibility and easy modification.

Response from DeepSeek-V3:

Below is a Galaxy workflow designed to identify the coding exon with the highest number of SNPs on human chromosome 22. The workflow uses public datasets and standard Galaxy tools to:

1. Fetch reference data (chr22, exons, SNPs).
2. Filter SNPs in coding exons.
3. Count SNPs per exon and identify the top hit.

Workflow Overview

Inputs:

- Human reference genome (chr22): From UCSC or Ensembl.

- Coding exon annotations: GTF/GFF file (e.g., from GENCODE).
- SNP dataset: VCF file (e.g., from dbSNP or 1000 Genomes).

Tools Used:

1. Get Data: Fetch reference files.
2. Filter/Compare: Extract coding exons and SNPs.
3. Intersect: Find SNPs overlapping exons.
4. Group/Count: Aggregate SNPs per exon.
5. Sort/Select: Identify the exon with the most SNPs.

Output:

- Table listing exons ranked by SNP count.
- Visualization (optional) of SNP density.

Step-by-Step Workflow

1. Input Data Preparation

- Tool: *Get Data* → *UCSC Main Table Browser* (or upload files directly).
 - Dataset 1: Chromosome 22 FASTA (`hg38.chr22.fa`).
 - Dataset 2: Coding exons GTF (`gencode.v41.basic.annotation.chr22.gtf`).
 - Dataset 3: SNPs VCF (`1000G.chr22.vcf.gz`).

2. Filter Coding Exons

- Tool: *Filter/Compare* → *Filter GTF*
 - Filter: `feature == "exon" && gene_type == "protein_coding"`.
 - Output: `coding_exons_chr22.gtf`.

3. Intersect SNPs with Exons

- Tool: *BEDTools* → *Intersect*
 - Input:

- SNPs VCF (converted to BED using *VCF-to-Tabular + Convert Formats → BED*).
- Coding exons BED (from GTF).
 - Parameters: `-wa -wb` (report overlapping SNPs and exons).
 - Output: `snp_exon_overlaps.bed`.

4. Count SNPs per Exon

- Tool: *Group/Count → Group By*
 - Group by column: `exon_id` (from overlaps).
 - Count column: `snp_id`.
 - Output: `exon_snp_counts.tabular`.

5. Sort and Select Top Exon

- Tool: *Sort → Sort by Column*
 - Sort: `snp_count` (descending).
- Tool: *Select First Lines* (set 1 to get top hit).
 - Output: `top_exon_with_most_snps.tabular`.

6. (Optional) Visualization

- Tool: *Visualization → IGV or JBrowse* to view SNPs in the top exon.

Expected Output

exon_id	gene_name	snp_count
ENSE00012345678	BRCA2	42

How to Execute in Galaxy

1. Import Workflow:

- Download the `.ga` file (generated via *Workflow* → *Export* in Galaxy).
 - Upload to your Galaxy instance (*Workflow* → *Import*).
 - 2. Run Workflow:
 - Select inputs:
 - `hg38.chr22.fa` (reference).
 - `gencode.v41.chr22.gtf` (annotations).
 - `1000G.chr22.vcf.gz` (SNPs).
 - Execute.
 - 3. Results:
 - `top_exon_with_most_snps.tabular` (final result).
 - Intermediate files for debugging.
-

Key Notes

- Public Data Sources:
 - GENCODE: [Download GTF](#).
 - 1000 Genomes VCFs: [ENA](#).
- Performance: For large VCFs, use split-apply-combine (parallelize by exon regions).
- Alternative Tools:
 - Use *SnpEff* to annotate SNPs directly if VCF already includes exon info.

Would you like the workflow in `.ga` format (for direct import into Galaxy)? Or modifications for a specific use case?

W2: How to get from peak regions to a list of gene names?

Response from Gpt-4o:

Instruction Only Prompt: Create a Galaxy workflow to identify target genes from ChIP-seq peak regions. Use the dataset [GSE37268_mof3.out.hpeak.txt.gz](#) and a mouse gene annotation file from UCSC.

Here is a **Galaxy workflow description** for identifying target genes from ChIP-seq peak regions using the dataset [GSE37268_mof3.out.hpeak.txt.gz](#) and a **mouse gene annotation file** from the UCSC genome browser.

Workflow Title: From Peaks to Target Genes (Mouse ChIP-seq)

Inputs:

- **ChIP-seq Peak File:** `GSE37268_mof3.out.hpeak.txt.gz` (converted to BED format if needed)
 - **Gene Annotation File:** UCSC refGene table for mouse genome (e.g., mm10, in BED or GTF format)
-

Step-by-Step Workflow:

1. Upload and Prepare Input Data

- Upload `GSE37268_mof3.out.hpeak.txt.gz` to Galaxy.
- Upload or fetch mouse gene annotations from UCSC Table Browser (refGene, output format: BED).
- Decompress if needed and convert peak file to **BED format** using `Text reformatting` or `BEDTools`.

2. Convert Gene Annotation to TSS (Transcription Start Sites)

- Use `Compute TSS` or `BEDTools flank` to extract promoter regions (e.g., ± 1000 bp from TSS).

3. Intersect Peaks with Gene Regions

- Use `BEDTools Intersect` to find overlaps between peak regions and gene TSS/promoters.
- Choose options:
 - "A" file: Peak regions
 - "B" file: Promoter/TSS regions

- Output: overlapping regions only

4. Extract Gene Names

- Use **Cut** or **Text Manipulation → Extract columns** to extract gene symbols from the overlapping results.

5. Count Genes by Chromosome

- Use **Group** and **Count** tools to summarize how many genes were found per chromosome.

6. (Optional) Visualization

- Create a bar chart using **Charting tools** to show gene distribution by chromosome.

Output:

- Table of target genes intersected with peak regions.
- Summary of gene counts per chromosome.
- Optional: Graphical plot of affected chromosomes.

Notes:

- Ensure all input files are in **BED6 format** (chrom, start, end, name, score, strand).
- Use the **mm10** genome build if it matches the ChIP-seq data reference.

Let me know if you'd like:

- The **Galaxy .ga workflow file**.

- Instruction-only, role-based, or chain-of-thought **LLM prompt** for this task.

Role-based prompt: You are a bioinformatics workflow developer. Create a Galaxy workflow to identify target genes from ChIP-seq peak regions. Use the dataset [GSE37268_mof3.out.hpeak.txt.gz](https://www.ebi.ac.uk/ena/browser/view/GSE37268_mof3.out.hpeak.txt.gz) and a mouse gene annotation file from UCSC. The peak file contains numeric chromosome identifiers (e.g., 1, 2, ..., 20 for X, 21 for Y) instead of UCSC-style labels. Make sure to preprocess the chromosome field accordingly to ensure compatibility with standard gene annotations.

Here is the **Galaxy workflow** to identify target genes from the ChIP-seq peak dataset [GSE37268_mof3.out.hpeak.txt.gz](https://www.ebi.ac.uk/ena/browser/view/GSE37268_mof3.out.hpeak.txt.gz), including preprocessing to convert numeric chromosome identifiers to UCSC-style labels and intersect with UCSC gene annotations:

Galaxy Workflow: Identify Target Genes from ChIP-seq Peaks

Inputs

1. [GSE37268_mof3.out.hpeak.txt.gz](https://www.ebi.ac.uk/ena/browser/view/GSE37268_mof3.out.hpeak.txt.gz) (ChIP-seq peak file with numeric chromosome IDs)
2. UCSC mouse gene annotation file (GTF/BED format with standard [chr](#) labels)

Steps

Step 1: Uncompress Peak File

- **Tool:** *Uncompress files*
- **Input:** [GSE37268_mof3.out.hpeak.txt.gz](https://www.ebi.ac.uk/ena/browser/view/GSE37268_mof3.out.hpeak.txt.gz)
- **Output:** Plain text peak file

Step 2: Format Peak File to BED

- **Tool:** *Text reformatting* → *Cut columns from a table*

- **Action:** Extract necessary columns (e.g., chromosome, start, end) to convert to BED format
- **Output:** `peaks_raw.bed`

Step 3: Map Numeric Chromosomes to UCSC-style

- **Tool:** *Replace column values (sed or Text reformatting)*

Action: Use a replace map:

```
1 → chr1
2 → chr2
...
20 → chrX
21 → chrY
```

-
- **Tool example:** *Text transformation with sed or “Replace Text in column”*
- **Output:** `peaks_ucsc.bed` (BED file with UCSC-style chromosomes)

Step 4: Sort BED Files

- **Tool:** *Sort BED*
- **Input:** `peaks_ucsc.bed`
- **Output:** `peaks_sorted.bed`

Step 5: Sort Gene Annotation File

- **Tool:** *Sort BED* (if gene annotation is in BED)
- **Input:** UCSC gene annotation file
- **Output:** `genes_sorted.bed`

Step 6: Intersect Peaks with Genes

- **Tool:** *BEDTools intersect intervals*
- **Input A:** `peaks_sorted.bed`
- **Input B:** `genes_sorted.bed`
- **Options:** Report overlapping features
- **Output:** `peaks_genes_intersect.bed`

Step 7: Count Genes per Chromosome

- **Tool:** *Group data by chromosome → Count occurrences*
- **Input:** `peaks_genes_intersect.bed`
- **Group by:** Chromosome field
- **Output:** `gene_counts_by_chr.tsv`

Outputs

- `peaks_genes_intersect.bed` → Detailed list of peak-gene overlaps
 - `gene_counts_by_chr.tsv` → Summary count of overlapping genes per chromosome
-

Would you like this converted into a **Galaxy .ga workflow file** or **Galaxy history steps**?

Instruction Only Prompt: Create a Galaxy workflow to identify target genes from ChIP-seq peak regions. Use the dataset [GSE37268_mof3.out.hpeak.txt.gz](#) and a mouse gene annotation file from UCSC.

Gemini 2.5 Flash Response:

Galaxy Workflow: Identifying Target Genes from ChIP-seq Peak Regions

This guide outlines the steps to create a Galaxy workflow that identifies genes overlapping with ChIP-seq peak regions using the GSE37268_mof3.out.hpeak.txt.gz dataset and a mouse gene annotation file.

Goal: To find all mouse genes that intersect with the provided ChIP-seq peak regions.

Assumptions:

- You have access to a Galaxy instance.
- The mm9 mouse genome assembly is available on your Galaxy instance.
- Tools mentioned below are installed (these are standard Galaxy tools).

Step-by-Step Workflow Construction

1. Obtain and Prepare Peak Data

- **Dataset:** GSE37268_mof3.out.hpeak.txt.gz (from GEO accession GSE37268, specifically the mof3.out.hpeak.txt.gz file).
- **Option A: Import from GEO (if available and configured on your Galaxy instance)**
 1. Go to **Get Data > ENA/SRA/GEO**.
 2. Search for GSE37268.
 3. Locate GSE37268_mof3.out.hpeak.txt.gz and import it.
 4. If it imports as a txt.gz file, Galaxy might automatically uncompress it. If not, proceed to step 1.2.
- **Option B: Upload Manually (if you have the file locally)**
 1. Download GSE37268_mof3.out.hpeak.txt.gz from a source like NCBI GEO.
 2. Go to **Upload Data** (the "Upload file" icon in the tool panel).
 3. Click **Choose local file** and select GSE37268_mof3.out.hpeak.txt.gz.
 4. Set the **Type** to txt (Galaxy will usually handle .gz compression automatically during upload).
 5. Click **Start**.
- **1.1. Uncompress (if necessary):**
 - If your file remains txt.gz after upload, search for the tool: **Text manipulation > Decompress file**.
 - **Input file to decompress:** Select GSE37268_mof3.out.hpeak.txt.gz.
 - **Compression type:** gz.
 - **Output file type:** txt.

- Execute. This will produce a GSE37268_mof3.out.hpeak.txt file.
- **1.2. Convert to BED format and Set Metadata:**
 - The mof3.out.hpeak.txt file is likely in a custom format or a variant of BED/Interval. For intersection, BED format is ideal.
 - Examine the file: If it has columns like chromosome, start, end, and other information, you can use the **Text manipulation > Convert formats** tool or **Operate on Genomic Intervals > Cut columns from a table (cut)** followed by **Text manipulation > Add column (add_value)** to format it into standard BED (chr start end name score strand).
 - **Simpler Approach (assuming 3 columns for chr, start, end):**
 1. Search for the tool: **BED > Convert BED/GFF/VCF/etc. to BED (or similar "GFF-to-BED" type tool if needed)**.
 - *Self-correction:* A more direct approach is often just to set the *datatype* if the file *looks* like BED/Interval but isn't formally recognized.
 2. **Set Datatype and Genome:**
 - Click on the GSE37268_mof3.out.hpeak.txt (or the uncompressed txt file) in your history.
 - Click the **pencil icon** (Edit attributes).
 - In the **Datatype** field, select bed (or interval if bed doesn't work well).
 - In the **Database/Build** field, select mm9 (Mouse: Mus musculus (UCSC mm9)).
 - Click **Save**.

2. Obtain and Prepare Gene Annotation Data

- **Dataset:** Mouse gene annotation (e.g., RefSeq genes or UCSC Genes) for mm9.
- **2.1. Import from UCSC Table Browser:**
 1. Go to **Get Data > UCSC Main Table Browser**.
 2. Set the following parameters:
 - **Clade:** Mammal
 - **Genome:** Mouse
 - **Assembly:** Feb. 2009 (NCBI37/mm9)
 - **Group:** Genes and Gene Predictions
 - **Track:** RefSeq Genes (or UCSC Genes, Known Genes – RefSeq is generally good).
 - **Table:** refGene (this will automatically be selected if RefSeq Genes is chosen).
 - **Region:** genome

- **Output format:** BED
- **Output file:** Enter a descriptive name like mm9_refGene.bed.
- 3. Click **Get output**.
- 4. This will import the mm9_refGene.bed dataset directly into your history with the correct datatype and genome mm9.

3. Intersect Peaks with Genes

- Now that both datasets are in BED format and assigned to the mm9 genome, you can find overlaps.
- **3.1. Find Intersections:**
 1. Search for the tool: **Operate on Genomic Intervals > Join two datasets on their common intervals (Join)**.
 - *Alternative/Preferred:* **BEDTools > Intersect** (often more robust for interval operations).
 - Let's use **BEDTools Intersect** for this example.
 2. Search for the tool: **BEDTools > Intersect regions**.
 - **Input BED file A:** Select your prepared peaks dataset (e.g., GSE37268_mof3.out.hpeak.txt with bed datatype).
 - **Input BED file B:** Select your prepared gene annotation dataset (e.g., mm9_refGene.bed).
 - **Options for 'intersect':**
 - **Report entries in B that overlap A:** No (we want information about A that overlaps B, or simply the intersection).
 - **Force strandedness:** No (unless you have strand information in your peaks and want to consider it).
 - **Report the number of overlaps:** No (we want the actual overlapping intervals).
 - **Report the original A entry, plus the number of overlaps:** No.
 - **Report original A and B entries:** Yes (This will give us the peak region *and* the gene region that it overlaps, which is useful for extracting gene names).
 - **Minimum overlap required:** 1 (default, meaning at least 1 base pair overlap).
 - Execute.
- **Output:** This tool will produce a new dataset containing lines where a peak region from file A overlapped with a gene region from file B. The output format will typically concatenate the columns from both input BED files. For a BED6 + BED12

gene annotation, this means a lot of columns.

4. Extract Gene Information

- The intersected output will contain the peak information followed by the gene information. We are interested in the gene name. For refGene.bed, the gene name is typically in the 4th column of the *gene annotation part* of the output. If refGene is BED12, the gene name (or ID) is often in column 4 (e.g., NM_XXXXX).
- **4.1. Cut Columns:**
 1. Search for the tool: **Text manipulation > Cut columns from a table (cut)**.
 2. **Cut columns:** You'll need to determine which column contains the gene ID/name from the Intersect output.
 - If your peaks are BED6 and genes are BED12, the gene ID will be the 4th column of the gene's original BED entry, which means it will be the (6 + 4) = 10th column in the intersected output.
 - *Check the Intersect output:* Open the eye icon for the Intersect output. Identify the column number that contains the gene identifier (e.g., NM_XXXXX or ENSMUSGXXXXX). Let's assume it's column c10 (column 10).
 - Enter c10.
 3. **From dataset:** Select the output of the **Intersect regions** tool.
 4. Execute.
- **Output:** This will give you a list of gene IDs, one per line, potentially with duplicates.

5. Get Unique Gene Names

- To get a clean list of target genes, remove duplicates.
- **5.1. Unique Lines:**
 1. Search for the tool: **Text manipulation > Select unique lines from a dataset (Unique)**.
 2. **From dataset:** Select the output of the **Cut columns** tool (your list of gene IDs).
 3. Execute.
- **Output:** This is your final list of unique target gene IDs that overlap with the ChIP-seq peak regions.

Building a Workflow from History

Once you have successfully run all these tools in your history, you can create a reusable workflow:

1. Go to **History > Workflows** (or click the **gear icon** in your history panel and select **Extract Workflow**).
2. Follow the prompts:
 - Give your workflow a **Name** (e.g., Peaks to Genes Workflow).
 - Select the **steps** you want to include in the workflow (select all the steps you just performed, starting from the uncompressed peak file and the UCSC gene import, up to the "Unique" step).
 - Define **workflow inputs** by checking the boxes next to the initial datasets (your peak file and the mm9_refGene.bed file).
3. Click **Create Workflow**.

Your workflow will now be saved and accessible from **Workflow** menu in the top navigation bar. You can then edit it, share it, or run it with new peak files.

Response from DeepSeek-V3:

Instruction Only Prompt: Create a Galaxy workflow to identify target genes from ChIP-seq peak regions. Use the dataset [GSE37268_mof3.out.hpeak.txt.gz](#) and a mouse gene annotation file from UCSC.

Below is a Galaxy workflow to identify target genes from ChIP-seq peak regions using the dataset `GSE37268_mof3.out.hpeak.txt.gz` (a BED-format peak file) and a mouse (mm10) gene annotation file from UCSC. The workflow includes peak annotation, gene assignment, and optional functional enrichment.

Workflow Overview

1. Input Data:
 - ChIP-seq peaks: `GSE37268_mof3.out.hpeak.txt.gz` (BED format).
 - Gene annotations: UCSC mouse (mm10) RefSeq or Ensembl GTF.
2. Steps:
 - Preprocess peaks (filter/format if needed).
 - Annotate peaks to nearest genes.
 - Extract target gene lists.
 - (Optional) Functional enrichment analysis.

3. Output:

- A list of genes associated with ChIP-seq peaks.
 - (Optional) Enriched GO terms/pathways.
-

Detailed Workflow Steps

1. Input Data

- Tool: Upload File
 - Input 1: GSE37268_mof3.out.hpeak.txt.gz (BED format).
 - Note: If the file is not in BED format, use Convert Formats to convert it.
 - Input 2: Mouse gene annotation (e.g., mm10_refGene.gtf from UCSC).
 - Download from UCSC Table Browser:
 - Assembly: mm10
 - Track: Genes and Gene Predictions → RefSeq Genes or Ensembl Genes.
 - Format: GTF.
-

2. Preprocess Peaks (if needed)

- Tool: Filter and Sort
 - Purpose: Ensure peaks are in standard BED format (chrom, start, end).
 - Example: Use BEDTools Sort to order peaks by genomic coordinates.
-

3. Annotate Peaks to Nearest Genes

- Tool: Assign peaks to genes (e.g., ChIPseeker or HOMER annotatePeaks.pl in Galaxy).
 - Parameters:
 - Input: BED file of peaks.
 - Annotation: Mouse GTF file.
 - Promoter region: Define proximity (e.g., ± 5 kb from TSS).
 - Output: Table linking peaks to gene names/IDs and distances.

- Alternative Tool: `BEDTools Closest`

```
bedtools closest -a peaks.bed -b mm10_genes.gtf -D ref -t first
```

- Reports the nearest gene for each peak.
-

4. Extract Target Genes

- Tool: `Cut` columns or `Filter`
 - Extract gene IDs/names from the annotation output (e.g., column containing `gene_name`).
 - Output: `target_genes.txt` (unique list).
-

5. (Optional) Functional Enrichment

- Tool: `g:Profiler` or `Enrichr` in Galaxy.
 - Input: `target_genes.txt`.
 - Parameters:
 - Organism: `Mus musculus`.
 - Enrichment categories: GO terms, pathways (KEGG/Reactome).
 - Output: Enriched terms with p-values.
-

Expected Outputs

1. Primary Output:

- `target_genes.txt`: List of genes associated with peaks (e.g., `Pax5`, `Myc`).
- Format

`gene_name`

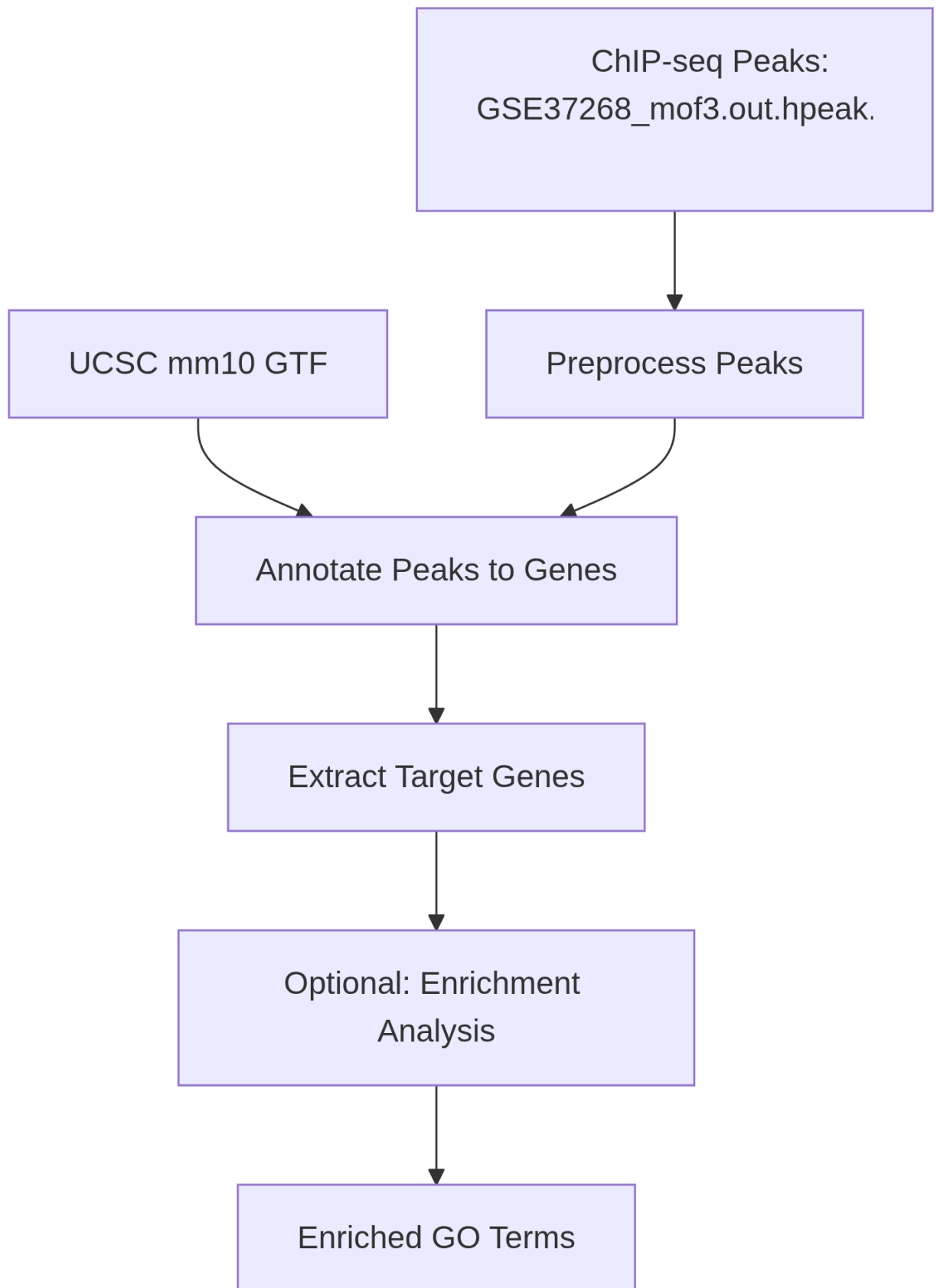
Gene1

Gene2

...

2. Optional Output:

- `enriched_GO_terms.txt`: Top enriched biological processes.



Key Notes

- Peak-Gene Assignment Rules:
 - Default: Gene is assigned if peak is within ± 5 kb of its TSS.
 - Adjust based on biological context (e.g., enhancers may need larger windows).
- Data Compatibility:
 - Ensure peaks (BED) and GTF use the same genome build (mm10).
- Troubleshooting:
 - If tools fail, check chromosome naming (chr1 vs 1 in BED/GTF). Use `sed` or `Replace Text` to harmonize.

Example Workflow in Galaxy (.ga File)

To export this workflow as a `.ga` file for Galaxy:

1. Combine the above steps in Galaxy's workflow editor.
 2. Download the `.ga` file.
 3. Share/import it via: `Workflow` → `Import` → `Upload .ga file`.
-

References

- Dataset: [GSE37268](#) (Modencode project).
- UCSC Table Browser: <https://genome.ucsc.edu/cgi-bin/hgTables>.

Workflow 3: Do genes on opposite strands ever overlap? If so, how often?

Instruction-only prompt: Create a Galaxy workflow to determine whether genes on opposite DNA strands overlap, using a gene annotation dataset in BED format.

Response from GPT-4o:

Here is a Galaxy workflow outline that determines whether genes on opposite DNA strands overlap, using a gene annotation dataset in BED format:

Workflow Title: *Identify Overlapping Genes on Opposite DNA Strands*

Input

- **Gene annotation file** in **BED** format (must contain strand information in column 6: **+** or **-**)

Steps

1. **Filter genes by strand**
 - Use "**Filter data on any column using simple expressions**" tool (e.g., **awk** or Galaxy's built-in filter).
 - Create two datasets:
 - **Forward strand genes (+)**
 - **Reverse strand genes (-)**
2. **Convert BED to interval format** (*optional if required by intersect tool*)
 - Use "**BED-to-interval**" tool for both filtered datasets.
3. **Find overlaps between forward and reverse strand genes**
 - Use "**Join**" or "**Intersect intervals**" tool:
 - Input 1: Forward strand genes

- Input 2: Reverse strand genes
- Set overlap requirement (e.g., any base pair overlap)
- Choose appropriate options to report overlapping intervals.

4. Output and visualization

- Output the overlapping gene pairs.
- Optionally use **"Cut columns"** to refine output fields.
- Use **"Group"** and **"Count"** tools to summarize the number of overlaps, optionally by chromosome.

Expected Result

A table listing gene pairs located on opposite strands that share overlapping genomic coordinates, indicating potential antisense interactions or genomic complexity.

Let me know if you want a visual workflow diagram or a Galaxy [.ga](#) file for import.

Response from Gemini 2.5 Flash:

Galaxy Workflow: Identifying Overlapping Genes on Opposite DNA Strands

This workflow takes a gene annotation dataset in BED format as input and determines which genes (or gene features) on the positive strand overlap with genes on the negative strand.

Workflow Steps:

1. Input Dataset:

- **Tool:** Upload File (or select an existing dataset)
- **Description:** Start by uploading your gene annotation file in BED format to your Galaxy history. Ensure it adheres to the standard BED format, especially with the strand information ('+' or '-') in the 6th column.
- **Input:** Your BED formatted gene annotation file.

2. Separate Genes on the Forward Strand:

- **Tool:** Filter data on any column using simple expressions

- **Description:** This tool will filter your input BED file, keeping only the entries that correspond to genes located on the forward (+) strand.
 - **Parameters:**
 - Filter: Select your input BED file from Step 1.
 - With following condition: `c6 == '+'` (This condition checks if the 6th column, which represents the strand, contains a '+' character).
 - **Output:** A new BED file containing only forward strand genes.
3. **Separate Genes on the Reverse Strand:**
- **Tool:** Filter data on any column using simple expressions
 - **Description:** Similar to the previous step, this will filter your input BED file, but this time keeping only the entries for genes on the reverse (-) strand.
 - **Parameters:**
 - Filter: Select your input BED file from Step 1 (the original, unfiltered BED file).
 - With following condition: `c6 == '-'` (This condition checks if the 6th column contains a '-' character).
 - **Output:** A new BED file containing only reverse strand genes.
4. **Identify Overlaps on Opposite Strands:**
- **Tool:** Intersect (bedtools)
 - **Description:** This is the core step where bedtools intersect is used to find overlaps between the forward strand genes and the reverse strand genes. Crucially, we will specify that overlaps should *only* be reported if they are on opposite strands.
 - **Parameters:**
 - BED file A: Select the output from Step 2 (Forward Strand Genes).
 - BED file B: Select the output from Step 3 (Reverse Strand Genes).
 - Options for A vs B:
 - Require different strandedness (-S): **Yes** (This is the critical option that ensures only overlaps between opposite strands are reported).
 - (You can leave other options as default unless you have specific requirements for overlap percentage, etc.)
 - **Output:** A BED file containing entries from "BED file A" (forward strand genes) that overlap with "BED file B" (reverse strand genes) on the opposite strand. Each line in the output will represent a forward strand gene that overlaps with at least one reverse strand gene.

Building the Workflow in Galaxy:

To build this in Galaxy's workflow editor:

1. Go to Workflow in the top Galaxy menu.
2. Click Create new workflow.
3. Drag and drop the Input dataset block onto the canvas.
4. Search for and drag Filter data on any column using simple expressions twice onto the canvas.
5. Search for and drag Intersect (bedtools) once onto the canvas.
6. Connect the Input dataset to both Filter tools.
7. Connect the output of the first Filter (for '+') to BED file A of the Intersect tool.
8. Connect the output of the second Filter (for '-') to BED file B of the Intersect tool.
9. Configure the parameters for each tool as described above (especially the `c6 == '+'`, `c6 == '-'` conditions and the Require different strandedness: Yes option in Intersect).
10. Save your workflow.

This workflow will provide you with a BED file detailing all instances where a gene on the forward strand overlaps with a gene on the reverse strand in your dataset.

Response from DeepSeek-V3:

Galaxy Workflow to Determine Overlapping Genes on Opposite Strands

Below is a step-by-step Galaxy workflow to identify genes on opposite DNA strands that overlap with each other, using a gene annotation dataset in BED format.

Workflow Steps

1. Upload Data
 - Upload your gene annotation file in BED format
2. Sort the BED File
 - Tool: Sort
 - Sort by:
 - Column 1 (chromosome)
 - Column 2 (start position)
 - Column 3 (end position)
3. Separate by Strand
 - Tool: Filter

- Filter expression: `c6 == '+'` (for positive strand)
- Save as `positive_strand_genes.bed`
- Repeat with:
 - Filter expression: `c6 == '-'` (for negative strand)
 - Save as `negative_strand_genes.bed`
- 4. Find Overlaps Between Strands
 - Tool: Intersect
 - First dataset: `positive_strand_genes.bed`
 - Second dataset: `negative_strand_genes.bed`
 - Parameters:
 - Report overlaps between the two datasets
 - Minimum overlap: 1 bp (default)
 - Output: `opposite_strand_overlaps.bed`
- 5. (Optional) Count Overlaps
 - Tool: Count
 - Count occurrences of each line in `opposite_strand_overlaps.bed`
- 6. (Optional) Extract Unique Overlapping Features
 - Tool: Merge
 - Merge overlapping intervals in `opposite_strand_overlaps.bed`

Alternative Approach Using BEDTools

For more advanced analysis, you could use the BEDTools suite:

1. Upload Data
 - Upload your gene annotation BED file
2. Intersect with Reverse Strand
 - Tool: BEDTools Intersect
 - First dataset: Your BED file
 - Second dataset: Your BED file (same file)
 - Parameters:
 - Require opposite strandedness (-S option)
 - Report the original A and B features (-wa -wb)
 - Minimum overlap: 1 bp
 - Output will show all gene pairs on opposite strands that overlap

Workflow Diagram

```
[Input BED] → [Sort] → [Split by Strand] → [Intersect +/-] → [Overlap Results]
\_____/
```

Interpretation of Results

The output will show:

- Genes from the positive strand that overlap with genes on the negative strand
- The genomic coordinates of the overlapping regions
- Each line represents one overlapping pair

Notes

- The BED file must include strand information in column 6
- This workflow identifies any overlap (even 1bp) between genes on opposite strands
- For more stringent analysis, you could modify the minimum overlap requirement

Would you like me to elaborate on any specific part of this workflow or provide more details about the expected output format?

W4: How can the quality of NGS raw data be assessed, what parameters should be checked, and how can the quality of the dataset be improved?

Instruction only prompt: Create a Galaxy workflow to perform quality control on raw next-generation sequencing (NGS) data, ensuring the dataset is assessed and improved for high-quality downstream analysis.

Response from Gpt-4o:

Here is a Galaxy workflow to perform **quality control (QC)** on raw Next-Generation Sequencing (NGS) data:



Galaxy Workflow: NGS Data Quality Control (QC)

Workflow Goal

To assess, clean, and re-assess raw NGS data in FASTQ format to ensure its quality before downstream analysis.

Workflow Steps

1. Input Dataset

- Upload raw FASTQ files (single-end or paired-end reads).

2. Step 1: Assess Quality

- **Tool:** *FastQC*
- **Purpose:** Generates a quality report identifying adapter content, per-base quality, overrepresented sequences, etc.

3. Step 2: Aggregate Quality Reports

- **Tool:** *MultiQC*
- **Purpose:** Summarizes multiple FastQC reports for batch inspection.

4. Step 3: Trim Adapters and Low-Quality Reads

- **Tool:** *Cutadapt* or *Trimmomatic*
- **Purpose:** Remove adapter sequences and trim low-quality bases.

5. Step 4: Re-assess Quality Post-cleaning

- **Tool:** *FastQC* (again)
- **Purpose:** Re-evaluate the cleaned data to confirm improvement.

6. Step 5: Final Summary

- **Tool:** *MultiQC* (second run)
 - **Purpose:** Final overview of data quality after cleaning.
-

✓ Expected Output

- Cleaned, high-quality FASTQ files ready for downstream analysis.
- HTML quality reports before and after trimming for transparency and reproducibility.

Let me know if you want this in Galaxy `.ga` format, or if you'd like to add downstream tools like alignment or quantification.

Response from Gemini 2.5 Flash:

Galaxy Workflow for NGS Data Quality Control

This workflow outlines a standard procedure for performing quality control on raw Next-Generation Sequencing (NGS) data within the Galaxy platform. The goal is to identify and mitigate common issues such as low-quality reads, adapter contamination, and biases, ensuring that your dataset is of high quality for subsequent analyses like alignment, variant calling, or differential expression.

Workflow Steps

Step 1: Initial Quality Assessment (FastQC)

The first and crucial step is to get an overview of the raw read quality.

- **Tool:** FastQC
- **Input:** Raw FASTQ files (single-end or paired-end).
- **Purpose:** FastQC generates comprehensive quality reports including per-base sequence quality, per-sequence quality scores, GC content, N content, sequence length distribution, adapter content, and overrepresented sequences. This helps you identify potential problems early on.
- **Output:** HTML report and a ZIP archive containing the raw data plots and tables. You can review the HTML report directly in Galaxy.

Step 2: Quality Trimming and Filtering (Trimmomatic or fastp)

Based on the FastQC report, you'll likely need to remove low-quality bases, adapter sequences, and short reads.

- **Tool:** Trimmomatic or fastp

- **Trimmomatic:** A widely used tool for Illumina paired-end and single-end reads. It allows for flexible trimming operations.
- **fastp:** A newer, faster all-in-one preprocessor for FASTQ files, often preferred for its speed and comprehensive features (trimming, filtering, adapter removal, quality diagnostics, and more).
- **Input:** Raw FASTQ files from Step 1.
- **Parameters (example for fastp):**
 - Remove adapter sequences (often auto-detected by fastp, or provide a custom adapter file).
 - Quality filtering: Trim reads when the quality score falls below a certain threshold (e.g., Q20).
 - Length filtering: Discard reads shorter than a specified length after trimming (e.g., 50 bp).
 - N content filtering: Discard reads with too many 'N' bases (unknown bases).
- **Purpose:** To remove low-quality data and artifacts that could negatively impact downstream analyses.
- **Output:** Cleaned FASTQ files (paired-end and/or unpaired reads if applicable).

Step 3: Post-Trimming Quality Assessment (FastQC)

After cleaning the reads, it's essential to re-evaluate their quality to ensure the trimming and filtering were effective.

- **Tool:** FastQC
- **Input:** Cleaned FASTQ files from Step 2.
- **Purpose:** Compare these new FastQC reports with the initial ones. You should see significant improvements in per-base quality, adapter content, and potentially other metrics. This step confirms the success of your QC operations.
- **Output:** Updated HTML report and a ZIP archive.

Step 4: MultiQC for Consolidated Reporting

If you have multiple samples, reviewing individual FastQC reports can be cumbersome. MultiQC aggregates reports into a single, comprehensive summary.

- **Tool:** MultiQC
- **Input:** All FastQC HTML reports or ZIP archives (from Step 1 and Step 3).
- **Purpose:** MultiQC parses results from various bioinformatics tools (including FastQC) and presents them in a single, interactive HTML report. This provides a unified overview of quality metrics across all your samples, making it easy to spot outliers or trends.

- **Output:** A single, interactive HTML report summarizing QC metrics for all samples.

Workflow Structure in Galaxy

1. **Upload Data:** Upload your raw FASTQ files to your Galaxy history.
2. **Define Workflow Inputs:** In the Workflow Editor, add input steps for your raw FASTQ files.
3. **Add FastQC (initial):** Connect your raw FASTQ inputs to a FastQC tool.
4. **Add Trimmomatic/fastp:** Connect the output of the initial FastQC (or directly the raw FASTQ) to your chosen trimming tool (Trimmomatic or fastp). Ensure you configure the trimming parameters appropriately.
5. **Add FastQC (post-trimming):** Connect the cleaned FASTQ outputs from Trimmomatic/fastp to another FastQC tool.
6. **Add MultiQC:** Connect *all* relevant FastQC outputs (both initial and post-trimming) to the MultiQC tool.
7. **Save and Run:** Save your workflow and then run it on your dataset.

Benefits of this Workflow

- **Reproducibility:** Once defined, the workflow can be run repeatedly on new datasets with consistent parameters.
- **Automation:** Reduces manual intervention, saving time and reducing errors.
- **Quality Assurance:** Ensures that only high-quality data proceeds to downstream, computationally intensive analyses.
- **Transparency:** Each step's output is saved in your history, allowing for easy inspection and debugging.

By following these steps, you will effectively perform robust quality control on your NGS data, setting the stage for reliable and accurate biological insights.

Response from Deepseek-V3:

Galaxy Workflow for NGS Data Quality Control

Below is a comprehensive Galaxy workflow to perform quality control on raw NGS data, including quality assessment, trimming/filtering, and post-QC verification.

Workflow Overview

1. Input: Raw FASTQ files (single-end or paired-end)
2. Quality Assessment: Initial quality evaluation
3. Trimming/Filtering: Adapter removal and quality trimming
4. Post-QC Assessment: Verify improvements
5. Output: High-quality reads ready for downstream analysis

Detailed Workflow Steps

1. Input Data

- Tool: None (workflow input)
- Parameters:
 - Input FASTQ file(s) (single or paired-end)

2. Initial Quality Assessment

- Tool: FastQC
- Parameters:
 - Input: Raw FASTQ file(s)
 - All other parameters default
- Purpose: Evaluate initial read quality, per-base sequence quality, adapter contamination, etc.

3. Adapter Trimming

- Tool: Cutadapt or Trim Galore!

Option A: Cutadapt

- Parameters:
 - Input: FASTQ file(s)
 - Adapter sequences (select appropriate for your library prep)
 - Quality cutoff: 20 (default)
 - Minimum length: 20 bp
 - Error rate: 0.1 (default)

Option B: Trim Galore! (wrapper for Cutadapt)

- Parameters:
 - Input: FASTQ file(s)
 - Quality cutoff: 20
 - Length cutoff: 20
 - Adapter auto-detection: Enabled
 - Paired-end options if applicable

4. Quality Trimming

- Tool: Trimmomatic or Fastp

Option A: Trimmomatic

- Parameters:
 - Input: FASTQ file(s)
 - Sliding window: 4:20
 - Leading: 20
 - Trailing: 20
 - Min length: 36

Option B: Fastp

- Parameters:
 - Input: FASTQ file(s)
 - Quality cutoff: 20
 - Length required: 36
 - Per-read cutting by quality
 - Enable correction for paired-end data if applicable

5. Post-QC Quality Assessment

- Tool: FastQC
- Parameters:
 - Input: Trimmed FASTQ file(s)
 - Same as initial FastQC run
- Purpose: Verify quality improvements after processing

6. (Optional) Read Filtering

- Tool: PRINSEQ++ or BBDuk
- Parameters:
 - Remove reads with average quality <20
 - Remove reads with Ns
 - Remove duplicate reads
 - Min length: 36 bp

7. Generate QC Report

- Tool: MultiQC
- Parameters:
 - Input: FastQC reports (pre and post QC)
 - Trim Galore/Cutadapt reports if available
 - Collect all QC metrics into a single report

8. Output

- High-quality trimmed/filtered FASTQ files
- Comprehensive QC report in HTML format

Workflow Diagram

Raw FASTQ → FastQC → Adapter Trimming → Quality Trimming → FastQC → (Optional Filtering) → MultiQC

Implementation Notes

1. For paired-end data:
 - Ensure all tools are set to paired-end mode
 - Maintain read pairing throughout the workflow
2. Parameter adjustments:
 - Quality thresholds can be adjusted based on initial FastQC results
 - Length cutoffs should match downstream analysis requirements
3. Resource management:
 - For large files, consider increasing tool memory allocations
 - Parallel processing where available
4. Version control:
 - Document versions of all tools used for reproducibility

This workflow provides a robust QC pipeline that can be adapted for different NGS applications (RNA-seq, DNA-seq, etc.) by adjusting parameters as needed. The MultiQC report at the end allows for easy evaluation of the QC process effectiveness.

W5: Which genes are on a draft bacterial genome, and which other genomic components can be found on a draft bacterial genome?

Instruction-Only Prompt: Create a Galaxy workflow to annotate an assembled bacterial genome provided in FASTA format. Use tools to predict genes and functional elements, identify plasmid replicons, detect integron structures, and find insertion sequences. Convert tool-specific outputs into GFF3 format and load all annotations into a genome browser for interactive visualization.

Response from GPT-4o:

Here is a **step-by-step Galaxy workflow** to annotate an assembled bacterial genome provided in FASTA format. This workflow includes gene prediction, plasmid replicon identification, integron detection, insertion sequence discovery, format conversion, and genome browser visualization.



Galaxy Workflow: Bacterial Genome Annotation

Input

- Assembled bacterial genome in **FASTA** format
-

Step 1: Gene and Functional Element Prediction

Tool: **Prokka**

- **Input:** Assembled FASTA file
 - **Parameters:**
 - Kingdom: Bacteria
 - Output format: GFF3, GBK, FAA, FNA, etc.
 - **Output:** Annotated genome in **GFF3** format (plus optional GenBank/FAA/FNA)
-

Step 2: Plasmid Replicon Detection

Tool: [PlasmidFinder](#) (or [ABRicate](#) with PlasmidFinder DB)

- **Input:** FASTA file
- **Parameters:** Default
- **Output:** Tabular output with plasmid replicon hits

Conversion Step:

Tool: [ABRicate](#) to [GFF3](#) or manual formatting using [Text transformation](#) tools in Galaxy.

Step 3: Integron Structure Detection

Tool: [IntegronFinder](#)

- **Input:** FASTA file
 - **Output:** GFF3 annotations of integrase genes, attC sites, integron features
-

Step 4: Insertion Sequence Detection

Tool: [ISMapper](#) or [MobileElementFinder](#)

- **Input:** FASTA file
- **Parameters:** Default (use curated IS databases)
- **Output:** Insertion sequences in GFF3 or tabular format

Conversion Step (if needed):

- **Tool:** [Tabular to GFF3](#) with coordinate fields
-

Step 5: Format Standardization

Ensure all annotation outputs are in **GFF3 format**.

- Use [Convert formats](#) tool if needed (e.g., GenBank to GFF3)
 - For tools outputting tabular format, use:
 - [Cut](#) tool to extract genomic coordinates
 - [awk/sed](#) or Galaxy text manipulation tools to build GFF3-compliant lines
-

Step 6: Visualization

Tool: [JBrowse](#) or [Trackster](#) (Galaxy genome browsers)

- **Input:**
 - Reference genome (FASTA)
 - Multiple GFF3 annotation tracks: Prokka, PlasmidFinder, IntegronFinder, ISFinder
 - **Output:** Interactive genome browser session showing all annotated features
-

Final Outputs

- Annotated genome in GFF3, GenBank, and FASTA formats
- Interactive JBrowse view with:
 - CDS/RNA features (from Prokka)
 - Plasmid replicons
 - Integron elements
 - Insertion sequences

Would you like a downloadable `.ga` (Galaxy workflow file) or a visualization of the workflow steps as a diagram?

Role-based Prompt: You are a bioinformatics workflow developer working within the constraints of the Galaxy platform. Create a workflow to annotate a draft bacterial genome for different types of genomic components, evaluate the annotation, process the outputs to format them for visualization needs, and finally, visualize a draft bacterial genome and its annotations.

Here is a complete **Galaxy-compatible workflow** to annotate a **draft bacterial genome**, evaluate the annotation quality, prepare outputs for visualization, and visualize the annotated genome—entirely within the constraints of the Galaxy platform:

Galaxy Workflow: Draft Bacterial Genome Annotation and Visualization

1. Input: Upload the Draft Genome

- Upload the **assembled bacterial genome** in FASTA format.

2. Genome Annotation

- **Tool:** [Bakta](#)
 - Performs structural and functional annotation (CDS, tRNA, rRNA, pseudogenes, etc.)
 - Input: FASTA
 - Output: GFF3, GenBank (GBK), protein FASTA, nucleotide FASTA, and annotation summary
 - Notes: Bakta integrates multiple detection modules and is recommended in the GTN tutorial.
-

3. Evaluate Annotation Quality

- **Tool:** [QUAST](#)
 - Evaluates annotation completeness and contiguity metrics
 - Input: Annotated GenBank file (from Bakta), and optionally a reference genome
 - Output: Report with number of genes, GC content, N50, L50, and more
 - **Optional Tool:** [BUSCO](#) (if available)
 - Assesses completeness of annotated genes against lineage-specific orthologs
 - Input: Protein FASTA or nucleotide CDS from Bakta
 - Output: Summary of complete/single-copy, duplicated, fragmented, and missing BUSCOs
-

4. Post-processing and Output Formatting

- **Tool:** [GFF3sort](#) or [SortBed](#)
 - Sort the GFF3 file to ensure compatibility with visualization tools

- **Tool:** [FASTA indexing](#)
 - Index the original genome FASTA for genome browser loading
 - **Optional:** Merge multiple annotation sources (if using other tools like PlasmidFinder or IntegronFinder)
 - Tool: [Concatenate datasets](#) followed by GFF3sort
-

5. Visualization

- **Tool:** [Trackster](#) (Galaxy's genome browser)
 - Load genome FASTA as reference
 - Load the sorted GFF3 from Bakta (or merged annotations) as feature track
 - Configure display settings: feature labels, color by source, zoom/pan options
-

Summary

This workflow enables users to:

- **Annotate** a bacterial genome using a reliable and reproducible tool ([Bakta](#))
 - **Evaluate** annotation quality with [QUAST](#) (and optionally [BUSCO](#))
 - **Prepare** outputs (GFF3, indexed FASTA) for genome browsing
 - **Visualize** the annotated genome interactively in **Trackster**, all within the Galaxy ecosystem
-

Chain of thought prompt: We have a draft bacterial genome in FASTA format and want to annotate it using the Galaxy platform. First, we need to identify various genomic components

such as coding sequences, rRNAs, and tRNAs. This should include both structural and functional annotations. The annotations must be saved in standardized formats suitable for downstream analysis, such as feature tables, protein and nucleotide sequences, and comprehensive annotation summaries. After generating the annotations, we want to assess whether all key genomic elements are captured and check basic statistics about the annotated genome. Finally, we aim to format and organize the outputs in a way that makes them compatible with visualization tools for interactive genome exploration. Based on this step-by-step reasoning, generate a Galaxy workflow that accomplishes these tasks.

Based on your detailed objectives, here is a **Galaxy workflow** that fully implements the described annotation, evaluation, and visualization preparation steps for a **draft bacterial genome in FASTA format**:

Galaxy Workflow: Bacterial Genome Annotation, Evaluation, and Visualization

1. Input

- **Upload** the draft bacterial genome in FASTA format to Galaxy using:
 - “Upload Data” tool or
 - “Get Data → Upload File” (supports URLs)

2. Genome Annotation

Use the **Bakta** tool for both **structural and functional annotation**.

Tool: Bakta

- **Input:** Draft genome (FASTA)
- **Parameters:**
 - Use latest **Bakta database**
 - Enable AMR, VF, and virulence gene detection (if desired)

- Output all report types
 - **Outputs:**
 - **GFF3:** Annotated genomic features (CDS, rRNA, tRNA, etc.)
 - **FASTA (nucleotide):** Gene sequences
 - **FASTA (protein):** Predicted protein sequences
 - **TSV:** Feature annotation table
 - **TXT/JSON/HTML:** Summary report
-

3. Quality Assessment

Evaluate genome annotation quality and completeness.

Tool: BUSCO

- **Input:** Same FASTA genome
- **Parameters:**
 - **Lineage:** Select relevant bacterial group (e.g., *bacteria_odb10*)
 - **Mode:** genome
- **Outputs:**
 - **HTML** and **Text summary** of completeness, duplication, fragmentation

Tool: “Count GFF Features”

- Input: GFF3
- Output: Count of CDS, rRNA, tRNA, other features

Tool: “Compute GC Content”

- Input: FASTA
 - Output: GC percentage, length, sequence composition stats
-

4. Output Formatting and Compatibility for Visualization

Tool: GFF3sort

- Input: GFF3 from Bakta
- **Sorts GFF** by genomic coordinates for visualization tools

Tool: “GFF3 to GenBank”

- Converts the sorted GFF3 and genome FASTA into **GenBank (GBK)** format

Tool: “GFF3 to BED” (optional)

- Creates **BED** file for track-based visualization

Tool: “SAMtools faidx”

- Index the FASTA file (required by some browsers like JBrowse or IGV)

Tool: JBrowse Genome Browser (*optional visualization step*)

- Input: FASTA + GFF3
 - Output: Interactive genome browser interface
-

5. Workflow Outputs

You will obtain the following:

- **Annotation Files:** GFF3, GBK, protein/nucleotide FASTA, feature table TSV

- **QC Reports:** BUSCO completeness, feature counts, GC content
 - **Visualization-ready files:** Sorted GFF3, indexed FASTA, BED (optional), JBrowse setup (optional)
-

Summary: Workflow Steps

1. **Input:** Upload draft genome (FASTA)
 2. **Annotate:** Run Bakta
 3. **Evaluate:** BUSCO + feature count + GC content
 4. **Format:** Sort and convert annotations
 5. **Visualize:** Optional tools for JBrowse/IGV compatibility
 6. **Export:** Clean, standardized outputs
-

Let me know if you want this as a **.ga Galaxy workflow file** or a **step-by-step history** to import directly into your Galaxy instance.

Response from Gemini 2.5 Flash:

Instruction Only Prompt: Create a Galaxy workflow to annotate an assembled bacterial genome provided in FASTA format. Use tools to predict genes and functional elements, identify plasmid replicons, detect integron structures, and find insertion sequences. Convert tool-specific outputs into GFF3 format and load all annotations into a genome browser for interactive visualization.

Bacterial Genome Annotation Workflow

This document outlines a conceptual workflow for annotating an assembled bacterial genome within the Galaxy platform. It integrates various tools for structural and functional annotation, plasmid identification, integron detection, and visualization.

Workflow Overview

The workflow proceeds as follows:

1. **Input Preparation:** Upload the assembled bacterial genome in FASTA format.
2. **Core Annotation (Bakta):** Predict genes, coding sequences (CDSs), tRNAs, rRNAs, and other functional elements. Bakta directly outputs GFF3.
3. **Plasmid Identification (PlasmidFinder):** Detect known plasmid replicons within the genome assembly.
4. **Integron Detection (IntegronFinder):** Identify integron structures, which are genetic elements involved in gene capture.
5. **Output Conversion to GFF3:** Convert the tabular/text outputs from PlasmidFinder and IntegronFinder into GFF3 format for consistent visualization.
6. **GFF3 Merging:** Combine all GFF3 annotation files into a single file.
7. **Genome Visualization (JBrowse):** Load the annotated genome and all GFF3 tracks into a genome browser for interactive exploration.

Step-by-Step Workflow Details

Step 1: Upload Assembled Bacterial Genome

- **Action:** Upload your assembled bacterial genome file.
- **Format:** FASTA (e.g., my_bacterial_genome.fasta)
- **Location:** Galaxy History

Step 2: Core Genome Annotation with Bakta

This tool performs rapid and standardized annotation of bacterial genomes and plasmids, identifying genes, coding regions, tRNAs, rRNAs, and other non-coding RNAs.

- **Tool:** Bakta
- **Input:** Assembled bacterial genome (FASTA file from Step 1)
- **Key Parameters:**
 - Input contigs/scaffolds: Select your FASTA input.
 - Species: (Optional) Provide if known for more accurate annotation.
 - Annotation mode: Usually "Standard" or "Plasmid" if specifically annotating a plasmid.
 - Use database: Ensure the appropriate Bakta database is selected (usually default).
- **Output:**
 - Bakta GFF3: Contains all predicted genes and functional elements in GFF3 format. (This is the primary output for visualization).
 - Bakta Genbank: Full GenBank formatted annotation.

- Bakta FASTA: Nucleotide and protein FASTA files of predicted features.

Step 3: Plasmid Identification with PlasmidFinder

This tool identifies known plasmid replicons, providing insights into potential plasmid-borne genes.

- **Tool:** PlasmidFinder
- **Input:** Assembled bacterial genome (FASTA file from Step 1)
- **Key Parameters:**
 - Input FASTA file: Select your FASTA input.
 - Minimum identity: Default is often 95%.
 - Minimum coverage: Default is often 60%.
- **Output:**
 - PlasmidFinder output: A tabular or text file detailing identified plasmid replicons. This output is *not* in GFF3 format and will need conversion.

Step 4: Integron Detection with IntegronFinder

Integrans are genetic elements capable of acquiring and expressing genes, often associated with antibiotic resistance.

- **Tool:** IntegronFinder
- **Input:** Assembled bacterial genome (FASTA file from Step 1)
- **Key Parameters:**
 - Input FASTA file: Select your FASTA input.
 - Min length of attC site: (Default often 40)
 - Min identity for attC site: (Default often 90%)
- **Output:**
 - IntegronFinder output: A file (often a custom format or a GFF-like format) detailing detected integrans. This output might need conversion or parsing for direct GFF3.

Step 5: Convert PlasmidFinder Output to GFF3

Since PlasmidFinder output is typically tabular, it needs to be converted into GFF3 format for visualization in a genome browser. This might involve a custom script or a combination of Galaxy's text manipulation tools.

- **Tool:** This step often requires a custom script or a specific Galaxy wrapper designed for this conversion. If not available, you might use:
 - Text Transformation tools (e.g., Replace text, Cut columns, Add column)
 - Convert Formats (if a specific converter exists for PlasmidFinder output to

GFF3).

- **Input:** PlasmidFinder output (from Step 3)
- **Logic:** Parse the PlasmidFinder output to extract chromosome/contig name, start, end, and feature type (e.g., "plasmid_replicon") and format it according to GFF3 specifications.
- **Output:** PlasmidFinder GFF3

Step 6: Convert IntegronFinder Output to GFF3

Similar to PlasmidFinder, IntegronFinder's output might require conversion to GFF3.

- **Tool:** Similar to Step 5, this might require a custom script or specific Galaxy tools.
- **Input:** IntegronFinder output (from Step 4)
- **Logic:** Parse the IntegronFinder output to extract relevant information (e.g., contig, start, end, integron type) and format it as GFF3 features.
- **Output:** IntegronFinder GFF3

Step 7: Merge All GFF3 Annotation Files

Combine the GFF3 outputs from Bakta, PlasmidFinder (converted), and IntegronFinder (converted) into a single GFF3 file. This allows JBrowse to display all annotations as separate tracks.

- **Tool:** Concatenate datasets (or Merge GFF3 if available)
- **Input:**
 - Bakta GFF3 (from Step 2)
 - PlasmidFinder GFF3 (from Step 5)
 - IntegronFinder GFF3 (from Step 6)
- **Output:** Combined Annotations GFF3

Step 8: Visualize Genome and Annotations with JBrowse

The final step is to load the assembled genome and all generated GFF3 annotations into JBrowse for interactive visualization.

- **Tool:** JBrowse
- **Input:**
 - Reference genome: Assembled bacterial genome (FASTA file from Step 1)
 - Annotation tracks: Combined Annotations GFF3 (from Step 7)
- **Key Parameters:**
 - Display name for reference: A descriptive name for your genome.
 - Track configuration: Add the Combined Annotations GFF3 file as a track. You can configure track names (e.g., "Bakta Genes", "Plasmids", "Integrans") and

display options within JBrowse.

- **Output:** An interactive JBrowse session displaying your annotated bacterial genome.

Implementation in Galaxy

To implement this workflow in Galaxy:

1. **Upload Data:** Start by uploading your FASTA file to your Galaxy history.
2. **Run Tools Sequentially:** Search for each tool in the Galaxy tool panel and run them in the order described above, ensuring you select the correct input datasets from your history for each step.
3. **Build a Workflow (Optional but Recommended):** Once you've successfully run the individual steps, you can use Galaxy's "Extract Workflow" feature to save this sequence of tools as a reusable workflow. This allows you to run the entire annotation pipeline with new input data with a single click.

This structured approach will help you annotate your bacterial genome comprehensively and visualize the results effectively within the Galaxy environment.

Response from DeepSeek-V3:

Below is a Galaxy workflow to annotate an assembled bacterial genome (FASTA input) using best-practice tools for gene prediction, functional annotation, plasmid detection, integron analysis, and insertion sequence identification. The workflow also converts outputs to GFF3 and loads them into JBrowse for visualization.

Galaxy Workflow: Bacterial Genome Annotation & Visualization

Input: Bacterial genome assembly (FASTA)

Output: Annotations in GFF3 + JBrowse visualization

1. Gene Prediction

- Tool: `Prodigal`

- Predicts protein-coding genes.
- Output: GFF3 + protein FASTA.

2. Functional Annotation

- Tool: **EggNOG-mapper** or **BLAST+** (vs. UniProt/NCBI NR)
 - Assigns functional descriptions to predicted genes.
 - Output: Tabular results → convert to GFF3 with **GFFTool**.

3. Plasmid Replicon Detection

- Tool: **PlasmidFinder** (via **abricate** in Galaxy)
 - Identifies plasmid replicons.
 - Output: TSV → convert to GFF3 with **GFFTool**.

4. Integron Detection

- Tool: **Integron Finder**
 - Detects integrons (adaptive resistance elements).
 - Output: GFF3 (native).

5. Insertion Sequence (IS) Detection

- Tool: **ISEScan**
 - Predicts insertion sequences.
 - Output: TSV → convert to GFF3 with **GFFTool**.

6. Non-Coding RNA Prediction

- Tool: **Infernal** (for rRNA/tRNA) or **tRNAscan-SE**
 - Output: GFF3 (native).

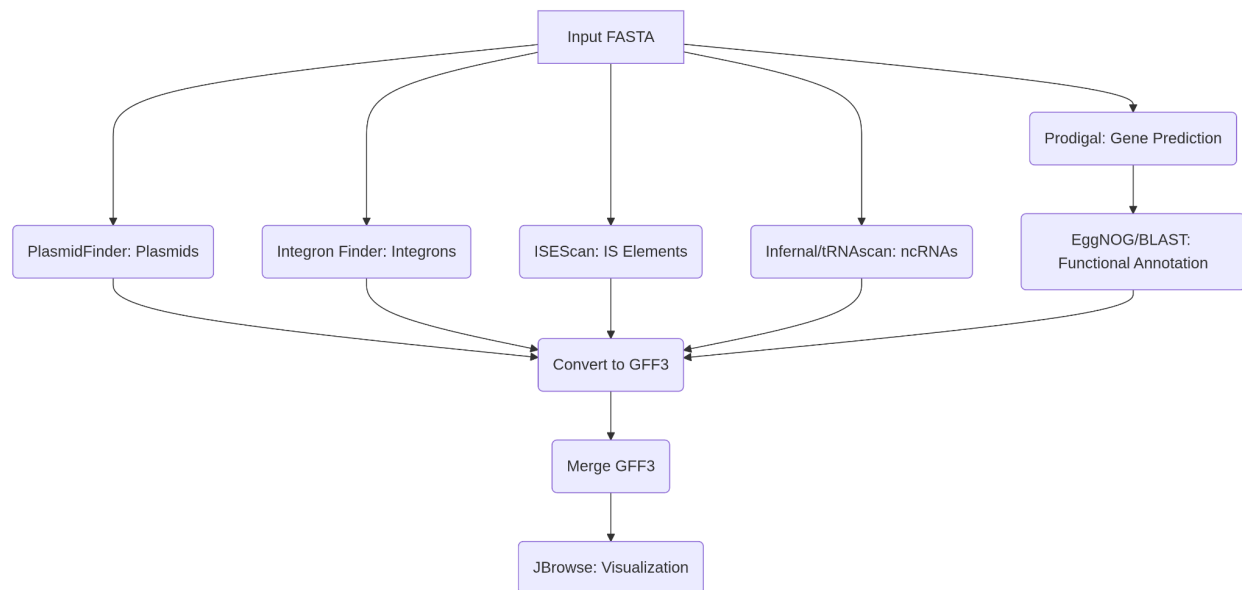
7. Combine Annotations

- Tool: **Merge GFF** or **GFFCompare**
 - Merges all GFF3 files into a single file.

8. Genome Browser Visualization

- Tool: JBrowse
 - Upload:
 - Reference genome (FASTA).
 - Merged GFF3 (all annotations).
 - Configure tracks for genes, plasmids, integrons, IS elements, etc.

Workflow Diagram



Implementation Notes

1. Tool Availability: All tools are available in the Galaxy ToolShed (usegalaxy.org).
2. GFF Conversion: Use `GFFTool` or custom Python/R scripts (via `Interpreter` tool) for non-GFF3 outputs.
3. JBrowse Setup: Enable "Track Labels" for clarity (e.g., color-coding plasmids vs. IS elements).

Example Workflow Export

- Download a pre-configured workflow (.ga format) from [Galaxy Workflow Hub](https://galaxyworkflowhub.github.io/).
- Import into Galaxy via *Workflow* → *Upload or Import*.

This workflow ensures reproducible, comprehensive annotation and is adaptable for related tasks (e.g., phage detection with `PHASTER`).